

DeepSpeed & ZeRO for LLMs

Train Bigger Models Faster & Cheaper

Practical solutions for overcoming the memory and computational challenges of training large language models at scale.



Why DeepSpeed?

Memory Constraints

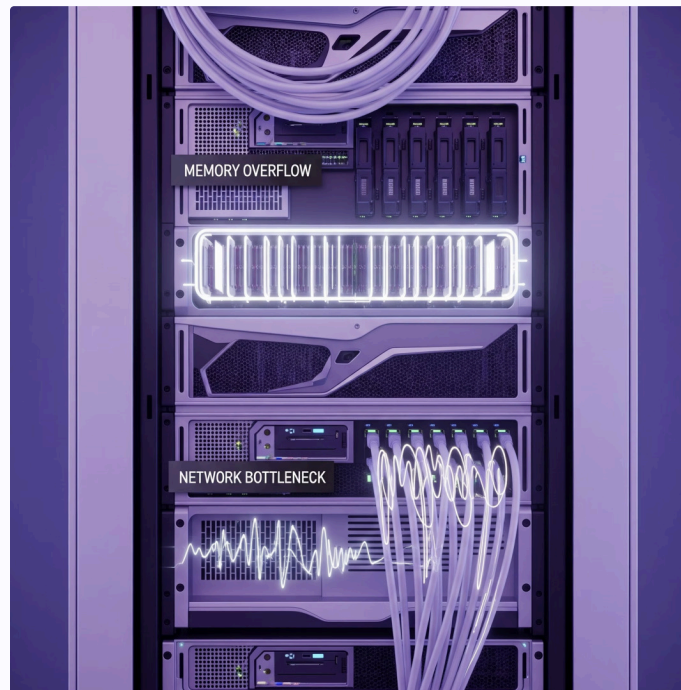
Memory limits block billion-parameter training on single GPUs

Storage Explosion

Checkpoint sizes become unmanageable at scale

Communication Bottlenecks

Network overhead significantly reduces training throughput



DeepSpeed addresses these challenges by reducing memory requirements, boosting training speed, and lowering overall cost.

ZeRO: Core Idea

Zero Redundancy Optimizer

Traditional data parallelism replicates the entire model across GPUs, wasting memory. ZeRO instead partitions:

Optimizer States

Adam moments, learning rates, etc.

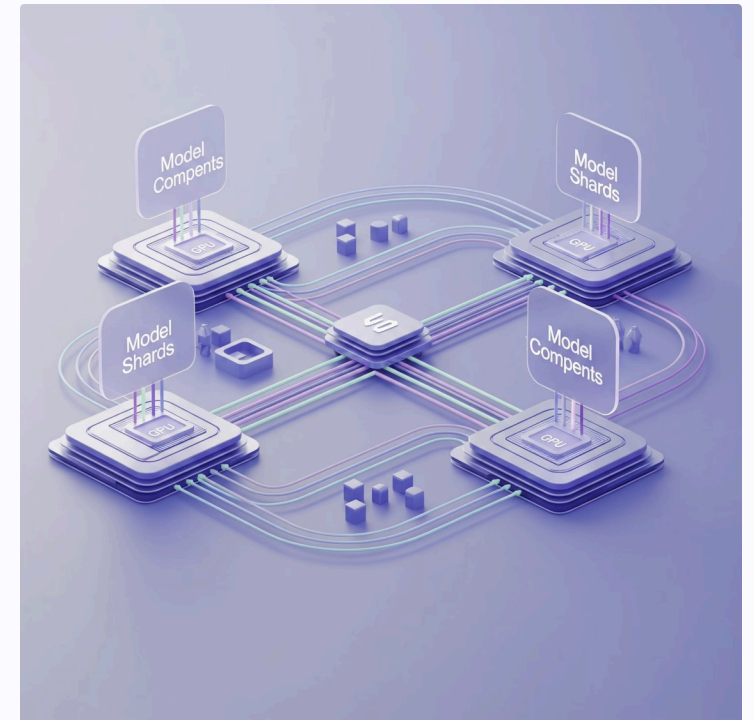
Gradients

Parameter updates during backprop

Parameters

Model weights themselves

By eliminating N-way redundancy, ZeRO achieves massive memory savings while preserving computational efficiency.



ZeRO Stages (0→3)

ZeRO implements progressive sharding in stages, each providing greater memory savings.

Stage 0: Baseline DDP

Standard Distributed Data Parallel with no sharding - full model replica on each GPU.

Stage 1: Shard Optimizer States

Distributes Adam optimizer states across GPUs, saving ~4x memory for these tensors.

Stage 2: + Shard Gradients

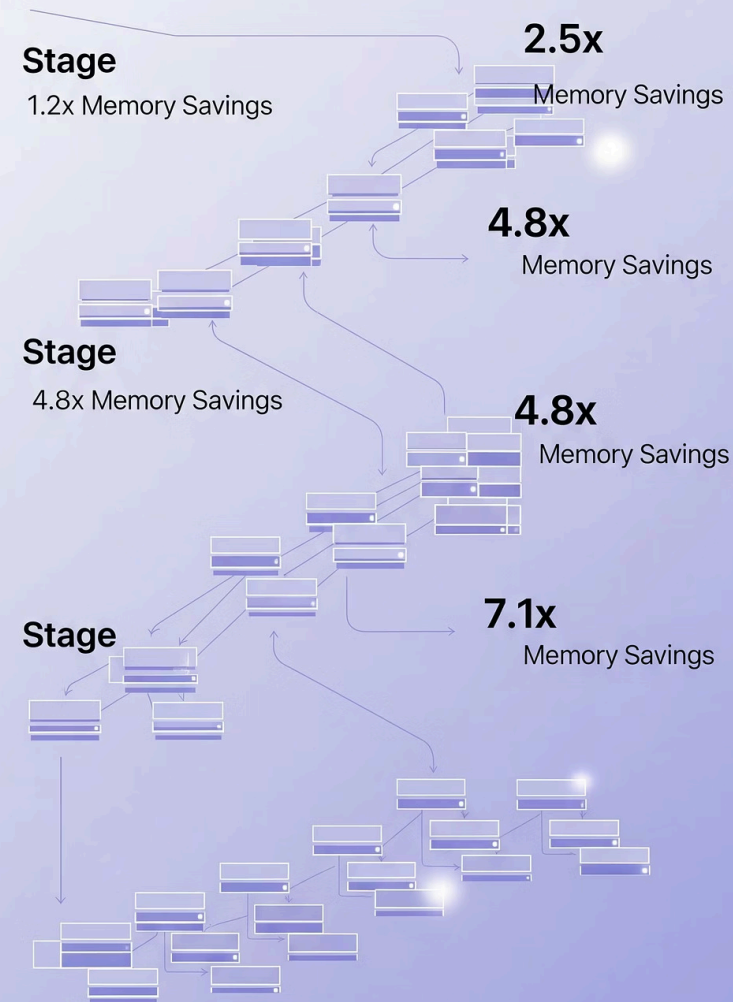
Additionally partitions gradients, eliminating their redundancy during backpropagation.

Stage 3: + Shard Parameters

Full sharding - distributes model parameters across GPUs, enabling training of massive models.

Result: Fit significantly larger batches and models on the same GPU hardware.

Model distribution of GPUs



Offloading Variants

ZeRO-Offload



Moves optimizer states and gradients to CPU RAM

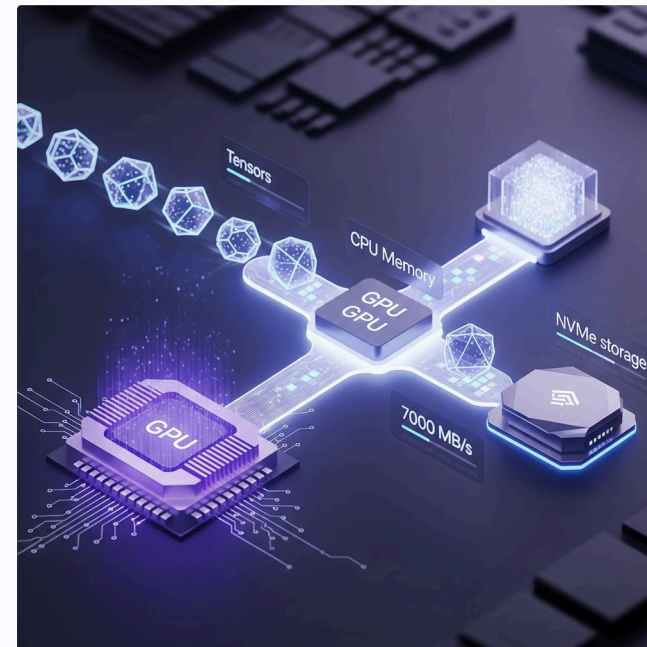
- Reduces GPU memory pressure
- Leverages abundant system memory
- PCIe transfer overhead

ZeRO-Infinity



Offloads model parameters and activations to CPU/NVMe

- Virtually unlimited model size
- Combines CPU RAM + NVMe storage
- Higher latency trade-off



The key to effective offloading is carefully tuning computation/communication overlap to hide latency penalties while maximizing memory savings.

Performance Techniques

Beyond memory optimization, DeepSpeed incorporates several techniques to maximize throughput:



Communication Overlap

Overlaps communication operations (reduce-scatter/all-gather) with computation to hide network latency during forward and backward passes.



Kernel Fusion

Combines operations like bias+GELU and layernorm into single GPU kernels to reduce memory transfers and kernel launch overhead.



Gradient Accumulation

Accumulates gradients over multiple micro-batches before optimizer steps, effectively emulating larger batch sizes with limited memory.



Activation Checkpointing

Trades computation for memory by discarding and recomputing activations during backprop, reducing peak memory usage.



DeepSpeed vs FSDP

DeepSpeed ZeRO-3

- Mature offloading capabilities to CPU/NVMe
- Advanced memory compression techniques
- Flexible training schedulers and optimizers
- Microsoft-supported, dedicated project
- Custom communication backend

PyTorch FSDP

- Native PyTorch API integration
- Stronger ecosystem compatibility
- Simpler setup within PyTorch workflows
- Community-driven development
- Uses PyTorch's native communication

Choose based on your specific offloading requirements, existing tooling ecosystem, and team familiarity with either framework.

Typical Config (Conceptual)

```
{
  "train_micro_batch_size_per_gpu": 4,
  "gradient_accumulation_steps": 8,
  "zero_optimization": {
    "stage": 3,
    "offload_optimizer": {
      "device": "cpu",
      "pin_memory": true
    },
    "offload_param": {
      "device": "none"
    },
    "overlap_comm": true,
    "contiguous_gradients": true
  },
  "bf16": {
    "enabled": true
  },
  "activation_checkpointing": {
    "partition_activations": true
  }
}
```

Key Config Elements

- ZeRO-3 with CPU offload
- Small per-GPU batch with accumulation
- BF16 mixed precision
- Activation checkpointing enabled
- Communication overlap activated

Adjust parameters based on your specific hardware configuration and model size requirements.

Practical Tuning Tips

Progressive Implementation

Start with ZeRO Stage 1, then move to 2, then 3. Verify stability at each stage before enabling offloading.

Measure Key Metrics

Track throughput (tokens/second) and memory headroom to identify bottlenecks and optimization opportunities.

Tune Communication Parameters

Optimize bucket sizes and all-gather/reduce-scatter overlap settings for your network topology.

Optimize Offloading

Pin CPU memory for faster transfers and use high-performance NVMe with proper queue depth for Infinity.



Finding the optimal configuration requires systematic experimentation with your specific hardware, model size, and training objectives.

Failure Modes & Debugging

OOM Errors Despite ZeRO

When you still hit out-of-memory errors despite ZeRO:

- Reduce micro-batch size
- Increase ZeRO stage (1→2→3)
- Enable offloading or activation checkpointing
- Check for memory leaks in custom code

Slow CPU/NVMe Offloading

When offloading creates severe bottlenecks:

- Verify PCIe bandwidth and utilization
- Check disk I/O performance and queue depth
- Enable pinned memory for CPU transfers
- Increase compute/communication overlap

Communication Issues

When training hangs or times out:

- Check NCCL debug logs for timeout errors
- Verify InfiniBand configuration
- Monitor network bandwidth utilization
- Test with smaller bucket sizes

Training Instability

When convergence fails with mixed precision:

- Review loss scaling approach
- Try BF16 instead of FP16 if available
- Check for NaN/Inf values in gradients
- Implement gradient clipping

When to Use What

<10B Parameters

On A100/H100 GPUs:

- ZeRO-2/3 + BF16 precision often sufficient
- Minimal offloading typically needed
- Focus on throughput optimization

>20B Parameters

Or constrained GPU environments:

- ZeRO-3 with aggressive offloading
- Consider NVMe extension for very large models
- Balance throughput vs. capacity trade-offs

Long Context Models

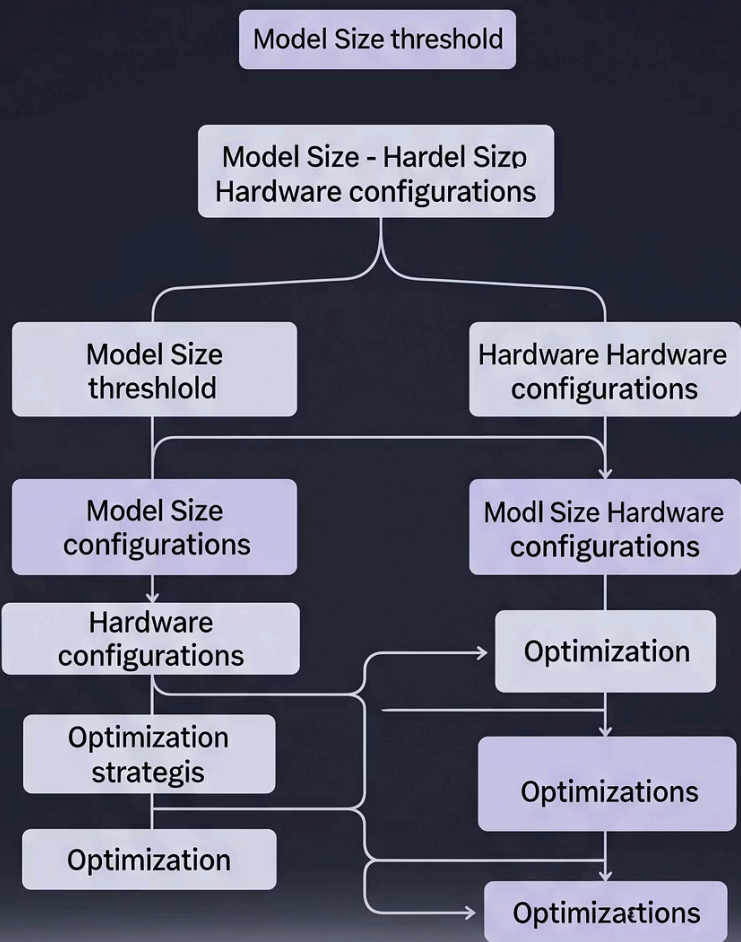
With substantial activation memory:

- Aggressive activation checkpointing
- Implement KV-cache optimization techniques
- Selective attention patterns

Multi-Tenant Clusters

Shared infrastructure environments:

- Prioritize elasticity and job recovery
- Optimize communication overlap
- Implement checkpoint/resume strategies



Key Takeaways

Memory Optimization

ZeRO shards optimizer states, gradients, and parameters to **remove redundancy** → fit bigger models on the same hardware

Capacity Extension

Offload techniques leverage **CPU RAM and NVMe storage** to extend capacity beyond GPU memory with latency trade-offs

Throughput Gains

Communication overlap and kernel fusion drive real training throughput improvements alongside memory savings

Iterative Tuning

Instrument, benchmark, and iterate—optimal configurations are highly specific to your hardware, network, and model architecture



📌 Getting the most from DeepSpeed requires balancing memory, computation, and communication optimizations for your specific workload.